



AIR UNIVERSITY
DEPARTMENT OF MECHATRONICS ENGINEERING
SEMESTER: SPRING 2026
Computer Programming
CE-112

Assignment-3

Instructor: Umer Farooq

Due Date: 7th May 2026

Soft Copy Time:10:00 PM

Cover Page must be filled as per attached.

Name: Faiz Khan

Name in ASCII(HEX): 46 61 69 7A 20 4B 68 61 6E

Rollno: 2501529

Rollno:in 4bit Hex 262B99

Question No	Write Status(Complete/Not complete and issue that you faced (compulsory Question will be checked on bases of this index)	Remarks Must be filled for every question	
1	Complete		
2	Complete		
3	Complete		
4	Complete		
5	Complete		

Q1
Code

```
#include <iostream>
using namespace std;
// Function to find min and max using
// reference parameters
void getExtremes(float &min, float &max,
float a[], int n) {
    min = a[0];
    max = a[0];
    for(int i=1; i<n; i++) {
        if(a[i] < min) {
            min = a[i];
        }
        if(a[i] > max) {
            max = a[i];
        }
    }
}

int main() {
    int n=5;
    float arr[] = { 4.5, 2.1, 8.9,
                    1.2, 5.5 };
    float minimum, maximum;
    getExtremes(minimum, maximum, arr, n);
}
```

```

    cout << "Array values: 4.5, 2.1, 8.9, 1.2, 5.5" << endl;
    cout << "Minimum value: " << minimum << endl;
    cout << "Maximum value: " << maximum << endl;

    return 0;
}

```

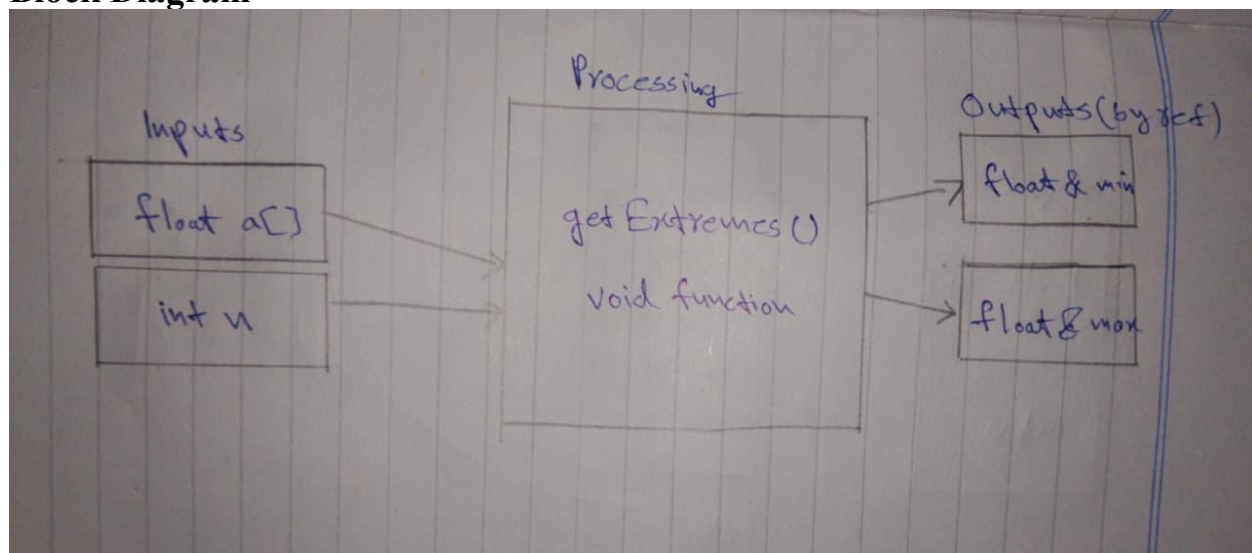
Output

```

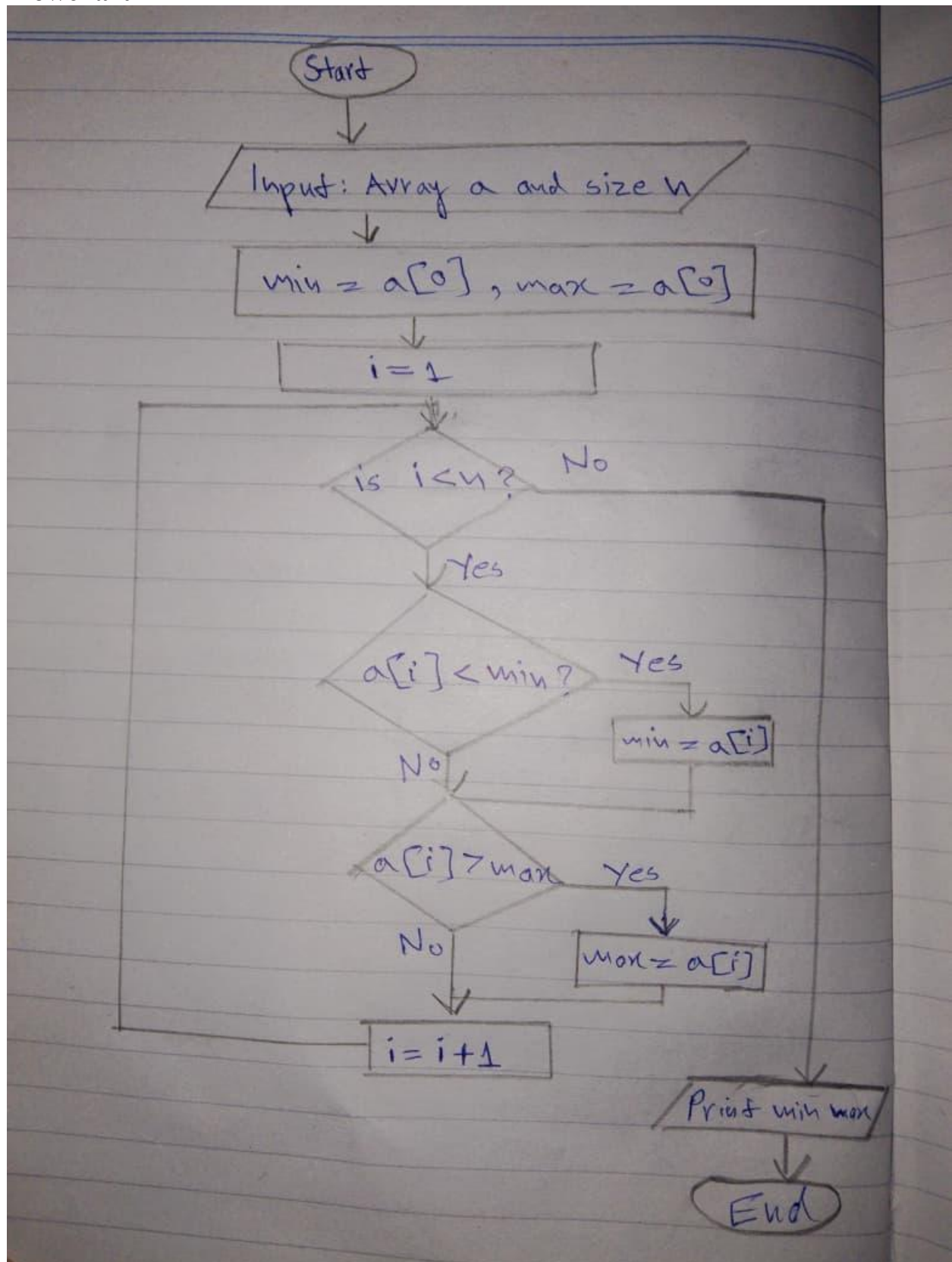
Array values: 4.5, 2.1, 8.9, 1.2, 5.5
Minimum value: 1.2
Maximum value: 8.9
PS C:\Users\dell\OneDrive\Desktop\C++\output>

```

Block Diagram



Flowchart



Q2 Code

```
#include <iostream>
#include <cmath> // for sqrt()
using namespace std;
double stdev(double x[], int n) { // Function
    // Step 1: Calculate mean (average)
    double sum = 0.0;
    for (int i = 0; i < n; i++)
        sum += x[i]; // Accumulate all values
    double mean = sum / n; // Compute mean
    // Step 2: compute sum of squared deviations
    double sqSum = 0.0;
    for (int i = 0; i < n; i++) {
        double diff = x[i] - mean; // deviation
        sqSum += diff * diff; // square and accumulate
    }
    // Step 3: Return square root of Variance
    return sqrt(sqSum / n);
    // sqrt(sum of sq deviations / n)
}

int main() {
    int n;
    cout << "Enter number of data\npoint: ";
    cin >> n;
```

```

double x[n];
cout << "Enter <n< Values: \n";
for (int i=0; i<n; i++)
    cin >> x[i];

cout << "Standard Deviation = " << stdev(n, n)
    << endl;
return 0;
}

```

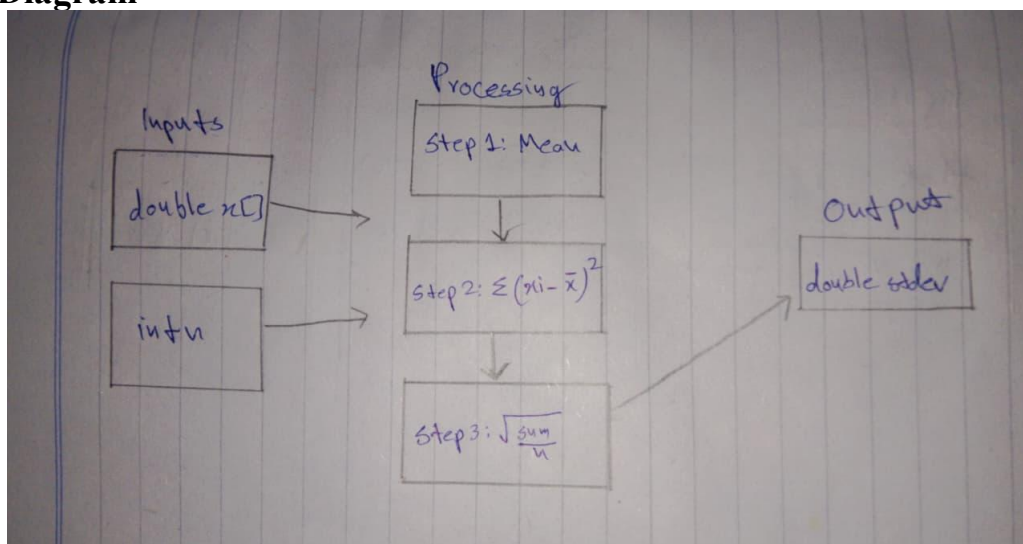
Output

```

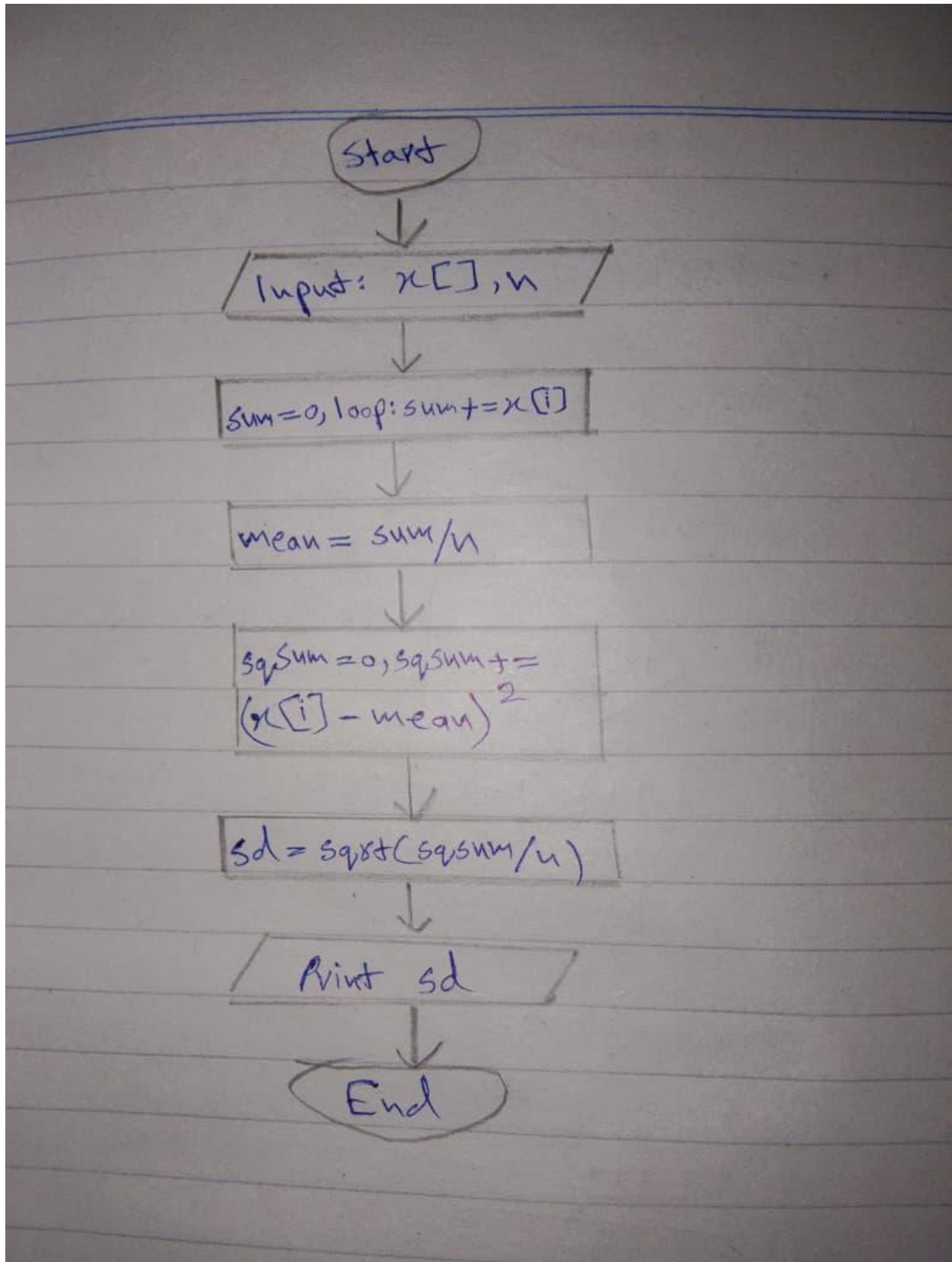
Enter number of data points: 7
Enter 7 values:
45
65
34
65
34
69
69
Standard Deviation = 14.9843
PS C:\Users\dell\OneDrive\Desktop\C++\output>

```

Block Diagram



Flowchart



Q3
Code

```
#include <iostream>
using namespace std;
// Function: get Value
// Reads a 3x3 matrix from user input
void getValue(int A[3][3], char name) {
    cout << "Enter values for matrix " << name
    << " (3x3): \n";
    for (int i=0; i<3; i++) // Row loop
        for (int j=0; j<3; j++) { // Column loop
            cout << "[" << i << "][" << j << "]= ";
            cin >> A[i][j];
        }
}

// Function: compute AXA
// Multiplies two 3x3 matrices A and B
void computeAXA(int A[3][3], int B[3][3],
    int C[3][3]) {
    for (int i=0; i<3; i++) {
        for (int j=0; j<3; j++) {
            C[i][j] = 0; // init result cell
            for (int k=0; k<3; k++)
                C[i][j] += A[i][k] * B[k][j];
            // Dot product
        }
    }
}
```



```

// Function: printArray
void printArray (int A[3][3], char name) {
    cout << "Matrix " << name << ": \n";
    for (int i=0; i<3; i++) {
        for (int j=0; j<3; j++)
            cout << A[i][j] << " ";
        cout << endl; // New Row
    }
}

```

```

int main() {
    int A[3][3], B[3][3], C[3][3];
    // Declare three matrices
    getValue (A, 'A'); // Read Matrix A
    getValue (B, 'B'); // Read Matrix B
    computeAXA(A, B, C); // Multiply:  $C = A \times B$ 

    printArray (A, 'A'); // Print Matrix A
    printArray (B, 'B'); // Print Matrix B
    printArray (C, 'C'); // Print Matrix C
    return 0;
}

```

Output

Enter values for Matrix A (3x3):

```
[0][0] = 4
[0][1] = 5
[0][2] = 6
[1][0] = 69
[1][1] = 69
[1][2] = 53
[2][0] = 34
[2][1] = 64
[2][2] = 34
```

Enter values for Matrix B (3x3):

```
[0][0] = 5
[0][1] = 5
[0][2] = 34
[1][0] = 456
[1][1] = 34
[1][2] = 12
[2][0] = 43
[2][1] = 23
[2][2] = 34
```

Matrix A:

```
4      5      6
69     69     53
34     64     34
```

Matrix B:

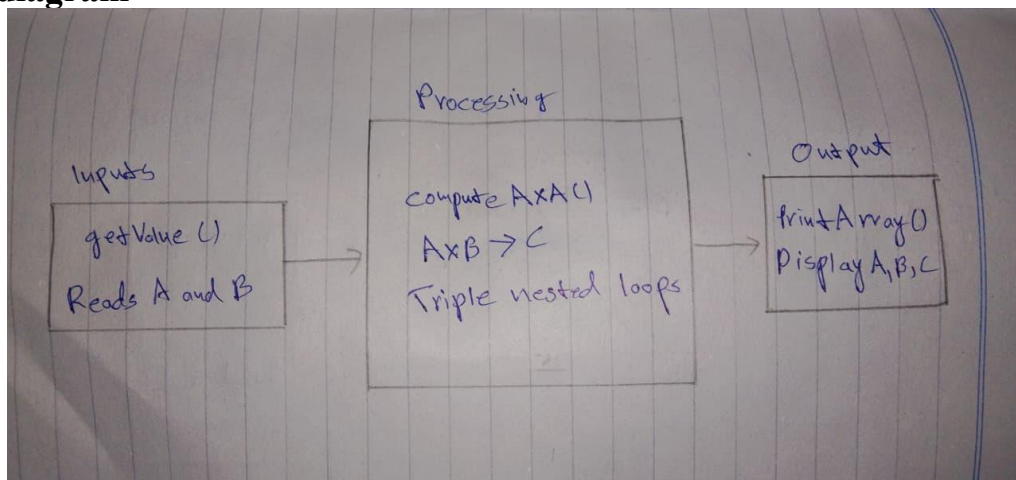
```
5      5      34
456    34     12
43     23     34
```

Matrix C:

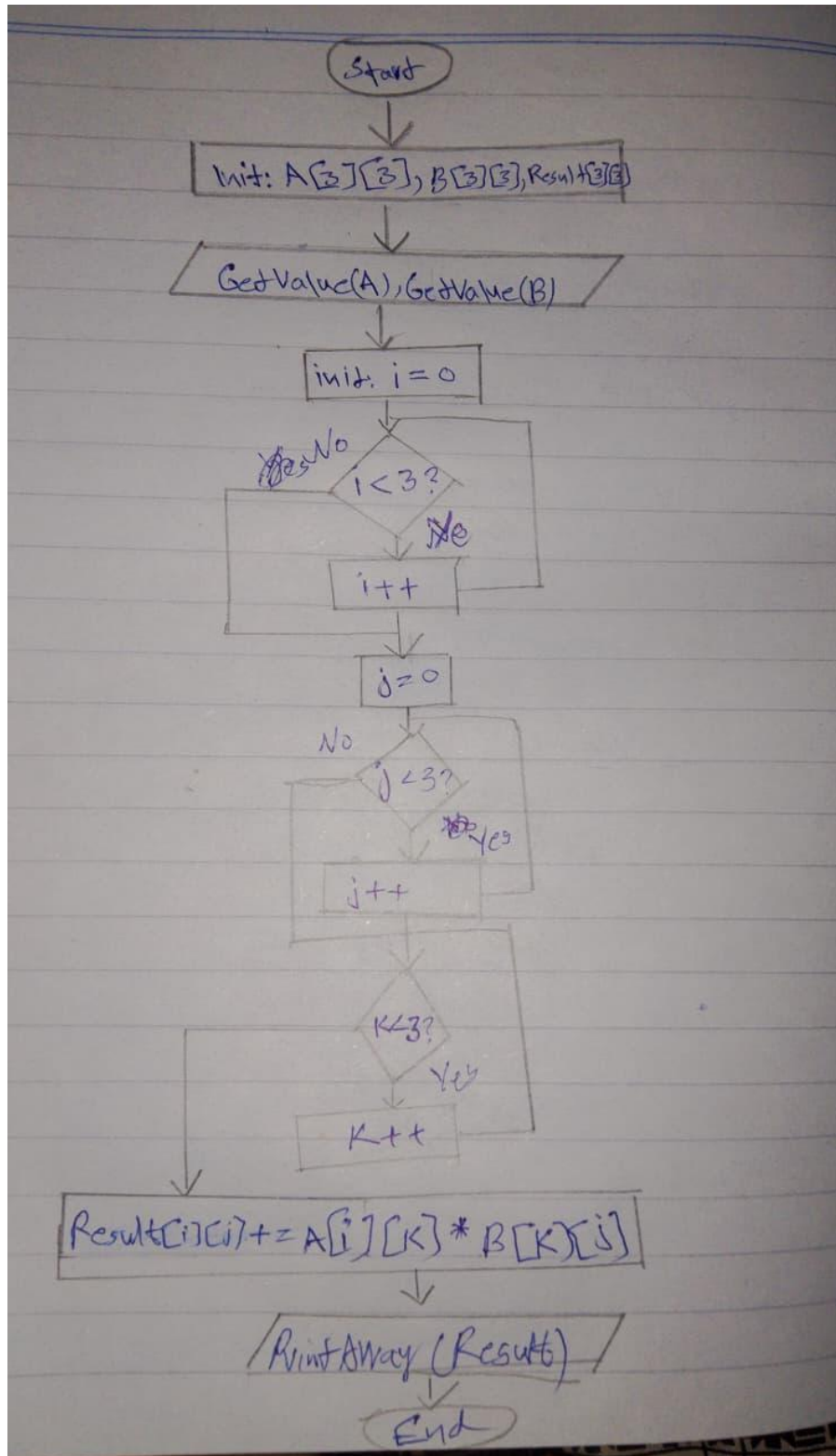
```
2558   328   400
34088  3910  4976
30816  3128  3080
```

PS C:\Users\dell\OneDrive\Desktop\C++\output>

Block diagram



Flowchart



Q4
Code

```
#include <iostream>
#include <string>
using namespace std;
// Function analyzeString
void analyzeString (const string & s) {
    int totalChars = s.length(); // total length
                                    // including all chars
    int spaces = 0; // count of space characters
    int tabs = 0; // count of tab characters
    int lines = 1; // At least 1 line
    // Traverse each character in the string
    for (int i = 0; i < totalChars; i++) {
        if (s[i] == ' ') spaces++; // space detected
        if (s[i] == '\t') tabs++; // tab detected
        if (s[i] == '\n') lines++; // Newline
    }
    // Display results
    cout << "Total Characters:" << totalChars
          << endl;
    cout << "Total Spaces:" << spaces << endl;
    cout << "Total tabs:" << tabs << endl;
    cout << "Total Lines:" << lines << endl;
}
```



```

int main() {
    string s;
    cout << "Enter a string (press enter when done): \n";
    getline(cin, s); // use getline to capture spaces & tabs
    analyzeString(s); // Call analysis function
    return 0;
}

```

Output

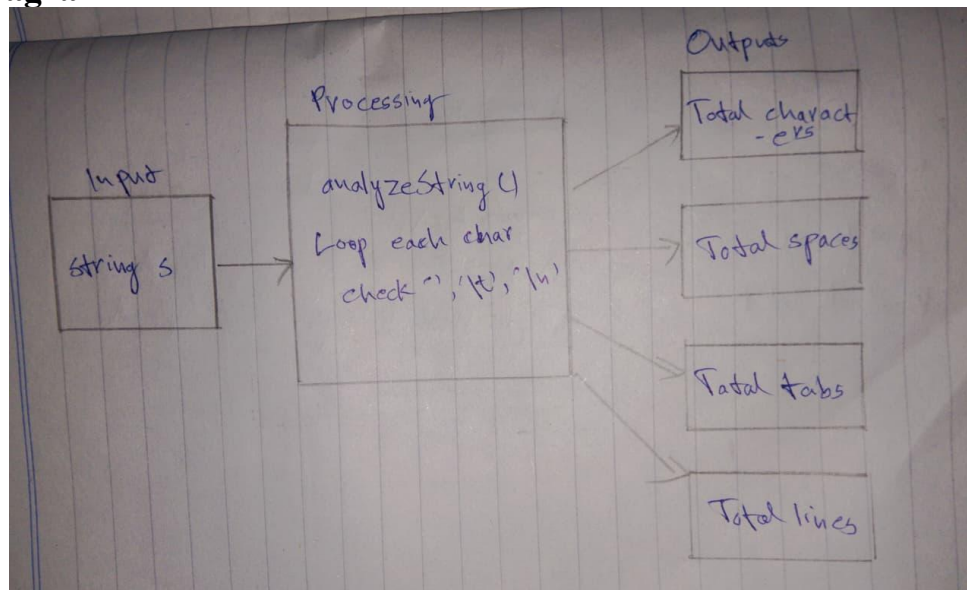
```

PS C:\Users\dell\OneDrive\Desktop\C++\output>
Enter a string (press Enter when done):

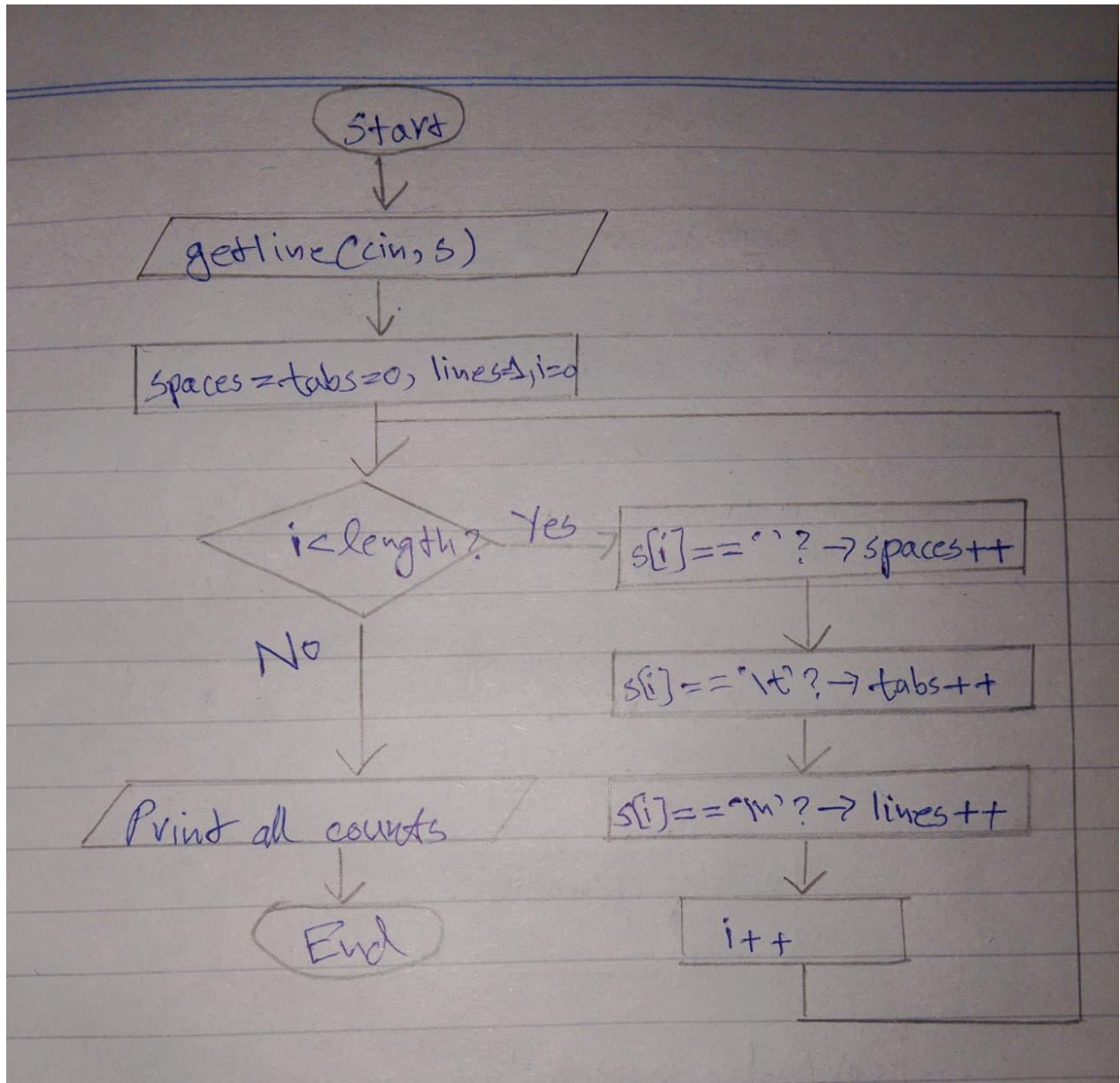
Total Characters : 3
Total Spaces      : 3
Total Tabs        : 0
Total Lines       : 1
PS C:\Users\dell\OneDrive\Desktop\C++\output>

```

Block diagram



Flowchart



Q5
Code

```
#include <iostream>
#include <algorithm> // for sort()
using namespace std;
// Function: calcMean
double calcMean(int a[], int n) {
    double sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i]; // sum all elements
    return sum/n; // Divide by count
}
// Function: calcMedian
double calcMedian(int a[], int n) {
    int temp[n];
    for (int i = 0; i < n; i++)
        temp[i] = a[i];
    // copy array to preserve original
    sort(temp, temp+n);
    // sort the copy in ascending order
    if (n%2 == 0) // Even count
        return (temp[n/2-1] + temp[n/2]) / 2.0;
        // Avg of two middles
    else
        return temp[n/2];
        // middle element (odd)
}
```


// Function: calcMode

```
int calcMode(int a[], int n) {  
    int mode = a[0]; // Assume first element  
                      // is mode
```

```
    int maxCount = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        int count = 0;
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (a[i] == a[j]) count++;
```

```
            // count occurrences of a[i]
```

```
        }
```

```
        if (count > maxCount) {
```

```
            // update if higher frequency
```

```
            maxCount = count;
```

```
            mode = a[i];
```

```
        }
```

```
    }
```

```
    return mode;
```

```
}
```



```

int main() {
    int n;
    cout << "Enter number of element: ";
    cin >> n;
    int a[n];
    cout << "Enter " << n << " integers: \n";
    for (int i = 0; i < n; i++) cin >> a[i];

    cout << "Mean = " << calcMean(a, n) << endl;
    cout << "Median = " << calcMedion(a, n) << endl;
    cout << "Mode = " << calcMode(a, n) << endl;
    return 0;
}

```

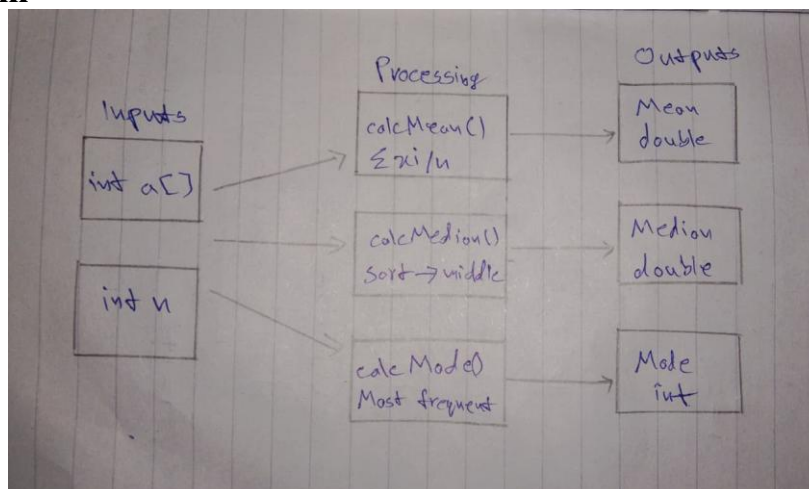
Output

```

PS C:\Users\dell\OneDrive\Desktop\C++\output> .\A
Enter number of elements: 5
Enter 5 integers:
69
75
69
420
805
Mean    = 287.6
Median  = 75
Mode    = 69
PS C:\Users\dell\OneDrive\Desktop\C++\output>

```

Block diagram



Flowchart

